

Chapter 22: Comparable and Comparator

Personal Notes

Sorting integers is easy — Java already knows how. But what about sorting your own Student objects by marks? Or by name? Or by age? Or by marks first, then name as a tiebreaker?

That's where Comparable and Comparator come in. These are the two interfaces Java uses to compare and sort custom objects. They're small but powerful — and they show up in every real-world Java codebase.

1. Why We Need These Interfaces

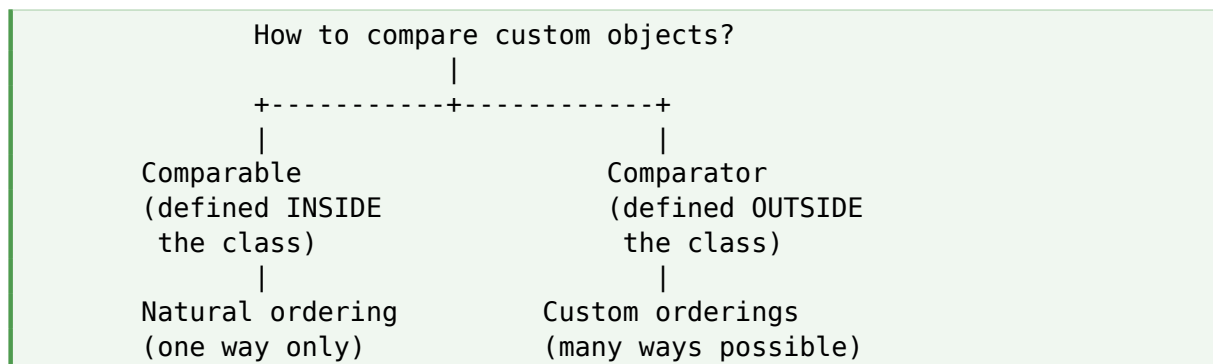
Sorting numbers and strings works out of the box — Java already knows $5 < 10$ and `"apple" < "banana"`. But what happens when you try to sort YOUR OWN objects?

```
List<Student> students = new ArrayList<>();
students.add(new Student("Aman", 92));
students.add(new Student("Riya", 88));
students.add(new Student("Karan", 95));
```

```
Collections.sort(students); // ERROR — Java doesn't know how!
```

Java doesn't know what "smaller" means for a Student. Smaller by marks? By name? By ID? It has no idea — and that is exactly the problem Comparable and Comparator solve.

The Two Approaches



2. The Comparable Interface

Comparable is a built-in Java interface used to define the NATURAL ORDERING of a class — the default way its objects should be compared and sorted.

The Interface Definition

```
public interface Comparable<T> {  
    int compareTo(T other);  
}
```

It has just ONE method: `compareTo()`. You implement it inside your class to teach Java how to compare your objects.

2.1 The `compareTo()` Return Value Rules

`compareTo()` returns an `int`, and the SIGN of that `int` decides the order:

Return Value	Meaning	Effect
Negative	<code>this < other</code>	this comes BEFORE other in sorted order
Zero	<code>this == other</code>	Order doesn't matter (they're equal)
Positive	<code>this > other</code>	this comes AFTER other in sorted order

Quick trick: For numeric fields, just return `(this.field - other.field)`. It gives negative, zero, or positive automatically — exactly what `compareTo()` wants. Even simpler: use `Integer.compare(a, b)` — it handles overflow safely.

2.2 Implementing `Comparable` on a Custom Class

```
public class Student implements Comparable<Student> {  
    String name;  
    int marks;  
  
    public Student(String name, int marks) {  
        this.name = name;  
        this.marks = marks;  
    }  
  
    @Override  
    public int compareTo(Student other) {  
        // Sort by marks (ascending)  
        return this.marks - other.marks;  
        // OR safer:  
        // return Integer.compare(this.marks, other.marks);  
    }  
  
    @Override  
    public String toString() {  
        return name + "(" + marks + ")";  
    }  
}
```

2.3 Using It

```
public class Main {
    public static void main(String[] args) {
        List<Student> students = new ArrayList<>();
        students.add(new Student("Aman", 92));
        students.add(new Student("Riya", 88));
        students.add(new Student("Karan", 95));

        Collections.sort(students);    // now works!

        System.out.println(students);
        // [Riya(88), Aman(92), Karan(95)]
    }
}
```

Why this works: Collections.sort() requires elements to be Comparable. Once Student implements Comparable<Student>, the sort method calls compareTo() repeatedly to figure out the right order. You provide the rule — Java does the work.

2.4 Descending Order

To sort in DESCENDING order, just swap the operands inside compareTo():

```
@Override
public int compareTo(Student other) {
    return other.marks - this.marks;    // descending
    // OR:
    // return Integer.compare(other.marks, this.marks);
}
```

3. The Comparator Interface

What if you want to sort the SAME class in MULTIPLE different ways? By marks today, by name tomorrow? That's where Comparator comes in.

Comparator is an EXTERNAL comparison strategy — defined OUTSIDE the class. You can have as many Comparators as you want for the same class, each with a different sorting rule.

The Interface Definition

```
public interface Comparator<T> {
    int compare(T a, T b);
}
```

It has one method: compare(a, b). The return value follows the same rules as compareTo().

3.1 Implementing Comparator as a Class

```
class SortByName implements Comparator<Student> {
    @Override
    public int compare(Student a, Student b) {
        return a.name.compareTo(b.name);
    }
}

class SortByMarks implements Comparator<Student> {
    @Override
    public int compare(Student a, Student b) {
        return a.marks - b.marks;
    }
}

// Usage:
Collections.sort(students, new SortByName());
Collections.sort(students, new SortByMarks());
```

Three separate comparators = three separate ways to sort the same Student list. The Student class doesn't need to change at all.

3.2 Comparator as a Lambda (Java 8+)

Creating a class just to write one comparison method is verbose. Java 8 lets you write Comparators inline using lambdas — way cleaner.

```
// Sort by marks ascending
Collections.sort(students, (a, b) -> a.marks - b.marks);

// Sort by name alphabetically
Collections.sort(students, (a, b) -> a.name.compareTo(b.name));

// Sort by marks descending
Collections.sort(students, (a, b) -> b.marks - a.marks);
```

3.3 Comparator.comparing() — Even Cleaner

Java 8 also added `Comparator.comparing()` — a static factory that takes a getter and produces a Comparator. It's the cleanest way to sort.

```
// Sort by marks ascending
students.sort(Comparator.comparing(s -> s.marks));

// Sort by name alphabetically
students.sort(Comparator.comparing(s -> s.name));

// Sort by marks descending
students.sort(Comparator.comparing((Student s) -> s.marks).reversed());
```

Reading these lines: `Comparator.comparing(s -> s.marks)` literally means "compare two Students by looking at their marks." Add `.reversed()` for descending. Much closer to plain English than any other form.

4. Comparator Chaining (Multi-Field Sort)

Real-world sorting often needs multiple criteria: "sort by marks, but if marks are equal, sort by name." Java makes this easy with `thenComparing()`.

```
students.sort(
    Comparator.comparing((Student s) -> s.marks)
                .thenComparing(s -> s.name)
);

// Result: primarily by marks. Ties broken by name alphabetically.
```

Multiple Levels

```
employees.sort(
    Comparator.comparing((Employee e) -> e.department)
                .thenComparing(e -> e.salary).reversed()
                .thenComparing(e -> e.name)
);
// Sort by department, then by salary descending, then by name
// alphabetically
```

Chain as many as you need. Each `thenComparing()` only kicks in when the previous one returned 0 (i.e., a tie).

5. Comparable vs Comparator — Quick Comparison

Aspect	Comparable	Comparator
Package	<code>java.lang</code>	<code>java.util</code>
Method	<code>compareTo(other)</code>	<code>compare(a, b)</code>
Defined where	INSIDE the class itself	OUTSIDE the class (separate class/lambda)
How many orders	Only ONE (natural ordering)	MANY — one Comparator per ordering
Modifies class?	Yes — class must implement it	No — works with any class
Usage	<code>Collections.sort(list)</code>	<code>Collections.sort(list, comparator)</code>
Best for	Default natural ordering	Custom or multiple orderings

Rule of thumb: Use Comparable to define the ONE most-natural way to sort a class (e.g., Students by ID). Use Comparator when you need DIFFERENT orderings — or when you can't modify the class (like a third-party library class).

6. Sorting Different Collections

6.1 Sorting a List

```
Collections.sort(list);                // natural ordering
// (Comparable)
Collections.sort(list, comparator);    // custom (Comparator)

// Or use the List's own sort method (Java 8+):
list.sort(null);                      // natural ordering
list.sort(comparator);                // custom
```

6.2 Sorting an Array

```
int[] nums = {5, 2, 8, 1};
Arrays.sort(nums);                    // natural ordering

Student[] arr = { ... };
Arrays.sort(arr);                     // uses Comparable
Arrays.sort(arr, comparator);         // uses Comparator
```

6.3 Auto-Sorted Collections

Some collections sort automatically as you add items — they require Comparable, or accept a Comparator in the constructor.

```
// TreeSet – sorted set
TreeSet<Student> set = new TreeSet<>();           // uses Comparable
TreeSet<Student> set2 = new TreeSet<>(comparator); // uses Comparator

// TreeMap – sorted by key
TreeMap<Student, Integer> map = new TreeMap<>(); // uses Comparable
TreeMap<Student, Integer> map2 = new TreeMap<>(comparator); // uses Comparator

// PriorityQueue – min-heap
PriorityQueue<Integer> pq = new PriorityQueue<>(); // natural
PriorityQueue<Student> pq2 = new PriorityQueue<>(comparator); // custom
```

Important: TreeSet and TreeMap REQUIRE either Comparable or a Comparator. If you try TreeSet<Student> without implementing Comparable AND without passing a Comparator, you'll get a ClassCastException at runtime.

7. Common Sorting Patterns

7.1 Reverse Order

```
// Method 1: Reverse a Comparator
students.sort(Comparator.comparing((Student s) -> s.marks).reversed());

// Method 2: Reverse natural ordering
Collections.sort(numbers, Comparator.reverseOrder());

// Method 3: Reverse after sorting
Collections.sort(list);
Collections.reverse(list);
```

7.2 Sorting Strings Case-Insensitive

```
names.sort(String.CASE_INSENSITIVE_ORDER);

// Or as lambda:
names.sort((a, b) -> a.compareToIgnoreCase(b));
```

7.3 Sorting by Field — Multiple Approaches

```
// Lambda
students.sort((a, b) -> a.marks - b.marks);

// Comparator.comparing
students.sort(Comparator.comparing(s -> s.marks));

// Comparator.comparingInt (more efficient for primitives)
students.sort(Comparator.comparingInt(s -> s.marks));
```

7.4 Null-Safe Sorting

```
// Put nulls first
students.sort(Comparator.nullsFirst(Comparator.comparing(s -> s.name)));

// Put nulls last
students.sort(Comparator.nullsLast(Comparator.comparing(s -> s.name)));
```

8. Full Example: Sorting in Multiple Ways

```
import java.util.*;

class Employee {
    String name;
    int age;
```

```

double salary;

public Employee(String name, int age, double salary) {
    this.name = name;
    this.age = age;
    this.salary = salary;
}

@Override
public String toString() {
    return name + "(" + age + ", " + salary + ")";
}
}

public class Main {
    public static void main(String[] args) {
        List<Employee> employees = new ArrayList<>();
        employees.add(new Employee("Aman", 28, 75000));
        employees.add(new Employee("Riya", 25, 90000));
        employees.add(new Employee("Karan", 28, 60000));
        employees.add(new Employee("Priya", 25, 90000));

        // 1. Sort by name
        employees.sort(Comparator.comparing(e -> e.name));

        // 2. Sort by salary (descending)
        employees.sort(Comparator.comparing((Employee e) ->
e.salary).reversed());

        // 3. Sort by age, then by salary (descending), then by name
        employees.sort(
            Comparator.comparingInt((Employee e) -> e.age)
                .thenComparing((Employee e) -> e.salary,
Comparator.reverseOrder())
                .thenComparing(e -> e.name)
        );

        for (Employee e : employees) {
            System.out.println(e);
        }
    }
}

```

9. Common Mistakes

Mistake 1: Subtraction overflow

```
return this.x - other.x;    // can OVERFLOW for very large/small ints!
```



```
// Safer:  
return Integer.compare(this.x, other.x);
```

If this.x is Integer.MIN_VALUE and other.x is positive, subtraction overflows and gives the wrong sign. Integer.compare() avoids this entirely.

Mistake 2: Inconsistent compareTo() and equals()

If compareTo() returns 0, equals() should normally return true. Mismatches cause weird bugs — especially in TreeSet and TreeMap, where the sorted collection uses compareTo() to detect duplicates.

Mistake 3: Forgetting @Override

@Override on compareTo() or compare() helps the compiler catch typos in the method signature.

Mistake 4: Using == with String inside compareTo()

```
return this.name == other.name ? 0 : ...    // WRONG – compares references  
return this.name.compareTo(other.name);    // correct for ordering
```

Mistake 5: Not casting in primitive lambdas

```
// May not compile without an explicit type hint:  
students.sort(Comparator.comparing(s -> s.marks).reversed());    // ERROR  
  
// Fix: tell Java what type s is  
students.sort(Comparator.comparing((Student s) ->  
s.marks).reversed());    // OK
```

Mistake 6: Using TreeSet without Comparable or Comparator

```
TreeSet<Student> set = new TreeSet<>();  
set.add(new Student("Aman", 92));    // RuntimeException – Student is not  
Comparable
```

Either implement Comparable in Student, or pass a Comparator to the TreeSet constructor.

10. Best Practices

- Implement Comparable if your class has ONE clear natural ordering (like Student by ID)
- Use Comparator for everything else — and prefer lambdas over separate classes
- Use Integer.compare() / Double.compare() instead of subtraction to avoid overflow
- Use Comparator.comparing() with thenComparing() for multi-field sorting — much cleaner
- Keep compareTo() consistent with equals(): if compareTo returns 0, equals should return true
- For descending order, prefer .reversed() over flipping the operands manually
- Use Comparator.nullsFirst() or nullsLast() if your data may contain nulls

11. Quick Reference

Concept / Method	Meaning
<code>Comparable<T></code>	Interface for natural ordering (one way to sort)
<code>compareTo(other)</code>	Method returning negative / zero / positive
<code>Comparator<T></code>	Interface for custom orderings
<code>compare(a, b)</code>	Method comparing two objects externally
<code>Collections.sort(list)</code>	Sort using Comparable
<code>Collections.sort(list, comp)</code>	Sort using Comparator
<code>Comparator.comparing(fn)</code>	Build a Comparator from a getter
<code>thenComparing(fn)</code>	Chain another Comparator for tie-breaking
<code>reversed()</code>	Reverse a Comparator
<code>Comparator.reverseOrder()</code>	Reverse natural ordering
<code>nullsFirst(comp) / nullsLast(comp)</code>	Handle null values safely
<code>Integer.compare(a, b)</code>	Overflow-safe int comparison

12. Practice Problems

Try these to lock in both interfaces.

1. Create a Student class with name and marks. Implement Comparable to sort by marks ascending.
2. Build a Book class with title, author, price. Sort a list of books by price ascending using Comparable.
3. Create a Movie class. Write a Comparator to sort by rating (descending), then by name (ascending).
4. Take a list of Strings. Sort them by LENGTH (shortest first) using a Comparator.
5. Sort a list of integers in descending order using Comparator.reverseOrder().
6. Create an Employee class. Write a single chained Comparator that sorts by department, then salary descending, then name.
7. Take a list of Strings with mixed case. Sort them case-insensitively.

8. Use `Comparator.comparing()` with method references (`Employee::getSalary`) to sort employees by salary.
9. Create a `TreeSet` of custom `Card` objects (with suit and value). Sort by value, then suit.
10. Sort a list of products by stock count, putting items with null stock at the end (use `nullsLast`).
11. Sort a list of `(firstName, lastName)` `Pair` objects by `lastName`, then by `firstName` for ties.
12. Write a `Comparator` that sorts numbers by absolute value (so `[-5, 3, -1, 7]` becomes `[-1, 3, -5, 7]`).

13. Final Takeaway

`Comparable` and `Comparator` are how Java teaches itself to sort YOUR data. `Comparable` is the "default rule" for a class — built in, one per class. `Comparator` is the "strategy pattern" for sorting — many ways, all external, swappable on demand.

In real-world Java code, you'll see `Comparable` used sparingly (only when there's an obvious natural ordering) and `Comparator` used everywhere — especially as lambdas and method references. Once these click, sorting any data the way you want becomes a one-liner.

Key Takeaway: `Comparable` = ONE way to sort, defined inside the class. `Comparator` = MANY ways to sort, defined outside. Use `Integer.compare()` instead of subtraction. Use `Comparator.comparing()` with `thenComparing()` for clean multi-field sorts. These three habits give you elegant sorting forever.

14. What's Next?

Now that sorting is solid, the next Java topics in your journey are:

- Exception Handling — `try`, `catch`, `finally`, `throw`, `throws`, custom exceptions
- Lambda Expressions & Functional Interfaces in depth
- Method References — the `::` operator for cleaner code
- Java Streams API — `filter`, `map`, `reduce`, `collect` on collections
- `Optional<T>` — handling "maybe null" safely
- Date and Time API (`java.time`)